STEP 4: CORE CS & TECHNICAL INTERVIEW QUESTIONS

The Rigorous Computer Science Fundamentals Grilling Guide

Evernorth Health Services Technical Round Preparation | Candidate: **Anshika Keshari**

Syllabus: Java OOP, DSA & DP, Operating Systems, DBMS/SQL, Computer Networks & Azure Cloud

1 Object-Oriented Programming (OOP in Java)

Computer Science Foundation: The 4 Pillars of OOP with Software Design Examples

1. **Encapsulation (Data Protection):**
   - *Concept:* Wrapping code and data into a single unit; keeping fields `private` and exposing controlled `public` getters/setters with boundary validation.
   - *Healthcare Example:* An `EHRRecord` class protects a patient's Social Security Number and medical diagnosis from being modified directly by rogue classes.
2. **Abstraction (Hiding Implementation Complexity):**
   - *Concept:* Showing only essential features to the outside world and hiding internal mechanics using abstract classes and interfaces.
   - *Example:* A `SecurityScanner` interface defines a `scanHost(String ip)` method. Whether the concrete implementation uses Nessus, Qualys, or a custom Python script is abstracted away.
3. **Inheritance (Code Reusability & Hierarchy):**
   - *Concept:* A derived class inherits properties and methods from a base class using `extends`.
   - *Example:* `RansomwareIncident` extends `SecurityIncident`, inheriting timestamp and severity fields while adding encryption key attributes.
4. **Polymorphism (One Interface, Multiple Implementations):**
   - **Compile-Time (Overloading):** Same method name, different argument signatures in the same class (e.g., `triggerAlert(String msg)` vs. `triggerAlert(String msg, int severity)`).
   - **Runtime (Overriding):** Subclass overrides parent method with `@Override`, resolved at runtime via Dynamic Method Dispatch.

Winning Technical Response: Frequently Asked Java Interview Questions

- **Q: What is the difference between an Interface and an Abstract Class in Java?** *Answer:* An Abstract Class can have instance state (non-static fields), constructors, and concrete methods. A class can extend only one abstract class (single inheritance). An Interface defines a pure behavioral contract (though Java 8+ allows default and static methods). A class can implement multiple interfaces (multiple inheritance of type).
- **Q: How does Java Memory Management work (Stack vs. Heap)?** *Answer:* Stack memory stores local primitive variables and method call execution frames (LIFO, fast, thread-private). Heap memory stores all instantiated objects (`new Object()`) accessible across threads and managed by the automatic Garbage Collector.

2 Data Structures & Algorithms (DSA & Your DP Camp)

Computer Science Foundation: High-Yield Data Structures for Technical Rounds

1. **HashMaps (Hash Tables):**
   - *Mechanism:* Computes `hash(key) % array_length` to store key-value pairs in buckets.
   - *Collisions:* Handled via Separate Chaining (LinkedList, transforming into a balanced Red-Black Tree in Java 8+ when a bucket exceeds 8 entries) or Open Addressing.
   - *Complexity:* Average $O(1)$ search, insert, and delete; worst-case $O(n)$ if all keys hash to the same bucket.
2. **Graph Algorithms:**
   - **Breadth-First Search (BFS):** Uses a `Queue` (FIFO). Traverses level-by-level. Finds shortest paths in unweighted graphs (e.g., finding the shortest network hop path between two compromised servers). Time: $O(V + E)$.
   - **Depth-First Search (DFS):** Uses a `Stack` or recursion. Explores down branch paths. Used for cycle detection, topological sorting, and path existence. Time: $O(V + E)$.
3. **Dynamic Programming (AlgoUniversity Camp Connection):**
   - *Core Idea:* Breaking a problem into overlapping subproblems and memoizing results to eliminate redundant computation.
   - **Top-Down (Memoization):** Recursive approach with cache lookup.
   - **Bottom-Up (Tabulation):** Iterative approach filling a DP table starting from base cases.
   - *Key DP Problems to Reference:* 0/1 Knapsack ($O(nW)$), Longest Common Subsequence ($O(nm)$), Coin Change ($O(nA)$).

3 Operating Systems (OS) & Concurrency

Computer Science Foundation: Processes, Threads & Deadlocks

1. **Process vs. Thread:**
   - A **Process** is an executing program instance with its own isolated virtual memory space (code, data, heap, stack) and OS handles.
   - A **Thread** is a lightweight execution stream within a process. Multiple threads share the process's code, data, and heap, but maintain private program counters and stacks.
2. **Deadlock & The 4 Coffman Conditions:** Deadlock occurs when processes are permanently blocked because each holds a resource needed by another. It requires all 4 conditions simultaneously:
   - **Mutual Exclusion:** Resources cannot be shared simultaneously.
   - **Hold and Wait:** A process holds allocated resources while waiting for additional ones.
   - **No Preemption:** Resources cannot be forcibly seized from a process.
   - **Circular Wait:** A circular chain where Process A waits for B, and B waits for A.
3. **Virtual Memory & Paging:** Allows executing programs larger than physical RAM. Virtual memory is split into fixed-size **Pages**; physical RAM is split into **Frames**. The Memory Management Unit (MMU) uses Page Tables for translation. If a requested page is absent from RAM, a **Page Fault** triggers swapping from disk.

4 DBMS & SQL (Healthcare Transaction Context)

Computer Science Foundation: ACID Properties & Indexing

1. **ACID Properties (Prescription Dispensing Example):** Updating a patient's prescription count and pharmacy inventory:
   - **Atomicity (All-or-Nothing):** Either both the patient record update and inventory deduction commit, or both rollback completely if a server crashes mid-flight.
   - **Consistency (Valid Rules):** Database constraints (e.g., inventory count ≥ 0) must never be violated.
   - **Isolation (Concurrency Safety):** Simultaneous requests accessing the same pharmacy record do not see each other's uncommitted intermediate states.
   - **Durability (Survival):** Committed updates are written to Write-Ahead Logs (WAL) and survive sudden power outages.
2. **Database Indexing (B+ Trees):**
   - Speeds up `SELECT` queries from an $O(n)$ full-table scan to an $O(\log n)$ balanced tree traversal.
   - *Trade-off:* Slows down `INSERT`, `UPDATE`, and `DELETE` operations because the B+ tree must maintain balance and split nodes.

5 Computer Networks & Azure Cloud Security

Computer Science Foundation: Networking & Cloud Fundamentals

1. **The TLS 1.3 Handshake (HTTPS Security):**
   - Client sends `ClientHello` with supported cipher suites and key share.
   - Server replies with `ServerHello`, certificate, and server key share.
   - Both derive a shared secret session key in just **1 round-trip (1-RTT)**, switching immediately to symmetric AES-256 encryption.
2. **Cloud Shared Responsibility Model (Microsoft Azure):**
   - **IaaS (VMs):** Cloud provider manages physical data center and hypervisor; Evernorth manages OS patches, network rules, and applications.
   - **PaaS (Azure SQL):** Provider manages OS and runtime; Evernorth manages data, access control, and application logic.
   - **SaaS (M365):** Provider manages everything; Evernorth manages user identity and access governance.
3. **Azure Key Vault & Role-Based Access Control (RBAC):** Enforces least privilege by binding specific security roles to Azure Active Directory identities and keeping encryption keys isolated from application source code.

Interview Trap & Edge Case: Interview Pro-Tip

Whenever asked a technical question about databases or operating systems, always give a **real-world healthcare example** (e.g., patient records, pharmacy transactions, cloud servers). It proves you think like an enterprise engineer!